

Name : Kanika Saini

SAP ID: 590034098

Program: Btech CSE B16

EXPERIMENT – 7

Implementation of Iterative and Decision-Making Constructs Using Modular Programming in C

1. Aim

To design, implement, compile, execute, and test a **menu-driven modular C program** that accepts a positive integer and performs different mathematical and digit-based operations using **functions, loops, selection statements, and fundamental data types**.

2. Problem Statement

Develop a menu-driven modular C project that accepts a **positive integer** and allows the user to perform the following operations:

1. Count the number of digits.
2. Find the sum of digits.
3. Find the product of digits.
4. Reverse the number.
5. Check whether the number is a palindrome.
6. Check whether the number is an Armstrong number.
7. Check whether the number is a Strong number.
8. Check whether the number is a Perfect number.
9. Check whether the number is Prime.
10. Find the largest digit.
11. Find the smallest digit.
12. Find GCD and LCM of two positive integers.
13. Exit.

The program must repeatedly display the menu using a **do-while loop** until the user selects Exit, and **switch-case** must be used to select the required operation.

3. Restrictions

According to the experiment, the following must **not** be used:

- Arrays
- Pointers
- Structures
- Unions
- Enumerations
- Recursion
- typedef
- pow() function

Only fundamental data types, functions, loops, and selection statements should be used.

4. Modular Structure of the Project

The complete program is divided into **four files** as specified in the experiment.

File	Responsibility
main.c	Reads input, displays menu and calls appropriate functions
digit_operations.c	Digit count, sum, product, reverse, largest and smallest digit
number_properties.c	Palindrome, Armstrong, Strong, Perfect and Prime tests
number_utils.c	Factorial, integer power, GCD and LCM
number.h	Header guard, constants and function declarations

5. Algorithms

Algorithm 1: Count the Number of Digits

Function: `int count_digits(int number)`

1. Start.

2. Set `count = 0`.
3. Repeat while the number is not zero using a `for` loop.
4. Divide `number` by 10.
5. Increment `count`.
6. Return `count`.
7. Stop.

This follows the required approach of repeatedly dividing the number by 10.

Test Cases

Test	Input	Expected Output	Actual Output	Result
Valid	12345	5 digits	5 digits	Pass
Boundary	1	1 digit	1 digit	Pass
Invalid	-25	Error – enter positive integer	Error – enter positive integer	Pass

6. Algorithm: Sum of Digits

Function: `int sum_of_digits(int number)`

1. Start.
2. Initialize `sum = 0`.
3. Extract the last digit using `number % 10`.
4. Add the digit to `sum`.
5. Remove the last digit using `number / 10`.
6. Repeat until the number becomes zero.
7. Return `sum`.
8. Stop.

The experiment specifically requires a `for` loop for this operation.

Test Cases

Test	Input	Expected Output	Actual Output	Result
Valid	1234	10	10	Pass
Boundary	1	1	1	Pass
Valid	555	15	15	Pass

7. Algorithm: Product of Digits

Function: `int product_of_digits(int number)`

1. Start.
2. Initialize `product = 1`.
3. Extract the last digit.
4. Multiply it with `product`.
5. Remove the last digit.
6. Repeat for all digits.
7. Return `product`.
8. Stop.

The product must initially be set to 1.

Test Cases

Test	Input	Expected	Actual	Result
Valid	1234	24	24	Pass
Boundary 1	1	1	1	Pass
Valid	105	0	0	Pass

8. Algorithm: Reverse the Number

Function: `int reverse_number(int number)`

1. Start.
2. Set `reverse = 0`.
3. Extract the last digit.
4. Calculate:

`reverse = reverse * 10 + digit`

5. Remove the last digit from the number.
6. Repeat until all digits are processed.
7. Return `reverse`.
8. Stop.

Test Cases

Test	Input	Expected	Actual	Result
Valid	1234	4321	4321	Pass
Boundary 1	1	1	1	Pass
Valid	1200	21	21	Pass

9. Algorithm: Palindrome Number

Function: `int is_palindrome(int number)`

1. Start.
2. Store the original number.
3. Call `reverse_number(number)`.
4. Compare the reversed number with the original.
5. If both are equal, return `TRUE`.
6. Otherwise, return `FALSE`.
7. Stop.

Test Cases

Test	Input	Expected Output	Actual Output	Result
Valid	121	Palindrome	Palindrome	Pass
Valid	123	Not Palindrome	Not Palindrome	Pass
Boundary 1	1	Palindrome	Palindrome	Pass

10. Algorithm: Armstrong Number

Function: `int is_armstrong(int number)`

1. Start.
2. Find the number of digits using `count_digits()`.
3. Extract each digit.
4. Calculate:

`digit^(number of digits)`

using `integer_power()`.

5. Add all calculated values.
6. Compare the sum with the original number.
7. If equal, return `TRUE`.
8. Otherwise, return `FALSE`.
9. Stop.

For example:

$$153 = 1^3 + 5^3 + 3^3 = 153$$

Therefore, 153 is an Armstrong number.

Test Cases

Test	Input	Expected	Actual	Result
Valid	153	Armstrong	Armstrong	Pass
Valid	123	Not Armstrong	Not Armstrong	Pass
Boundary 1		Armstrong	Armstrong	Pass

11. Algorithm: Strong Number

Function: `int is_strong(int number)`

1. Start.
2. Initialize `sum = 0`.
3. Extract every digit.
4. Find its factorial using `factorial()`.
5. Add the factorial to `sum`.
6. Repeat for every digit.
7. Compare `sum` with the original number.
8. If equal, return `TRUE`.
9. Otherwise, return `FALSE`.
10. Stop.

Example:

$$145 = 1! + 4! + 5!$$

$$= 1 + 24 + 120$$

$$= 145$$

Hence, 145 is a Strong number.

Test Cases

Test	Input	Expected	Actual	Result
Valid	145	Strong	Strong	Pass
Valid	123	Not Strong	Not Strong	Pass
Boundary 1		Strong	Strong	Pass

12. Algorithm: Perfect Number

Function: `int is_perfect(int number)`

1. Start.
2. Initialize `sum = 0`.
3. Find all proper divisors of the number.
4. Add the proper divisors.
5. Compare their sum with the original number.
6. If equal, return `TRUE`.
7. Otherwise, return `FALSE`.
8. Stop.

Example:

$$28 = 1 + 2 + 4 + 7 + 14$$

Therefore, 28 is a Perfect number.

Test Cases

Test	Input	Expected	Actual	Result
Valid	28	Perfect	Perfect	Pass
Valid	12	Not Perfect	Not Perfect	Pass
Boundary 1		Not Perfect	Not Perfect	Pass

13. Algorithm: Prime Number

Function: `int is_prime(int number)`

1. Start.
2. If `number < 2`, return `FALSE`.
3. Start checking divisibility from 2.
4. Continue while:

`divisor <= number / divisor`

5. If the number is divisible by any divisor, return `FALSE`.
6. Otherwise, return `TRUE`.
7. Stop.

Test Cases

Test	Input	Expected	Actual	Result
Valid	7	Prime	Prime	Pass
Valid	10	Not Prime	Not Prime	Pass
Boundary	1	Not Prime	Not Prime	Pass

14. Algorithm: Largest Digit

Function: `int largest_digit(int number)`

1. Start.
2. Initialize `largest` with the last digit.
3. Remove the last digit.
4. Extract each remaining digit.
5. Compare the digit with `largest`.
6. If the digit is greater, update `largest`.
7. Repeat for all digits.
8. Return `largest`.
9. Stop.

Test Cases

Test	Input	Expected	Actual	Result
Valid	58321	8	8	Pass
Valid	111	1	1	Pass
Boundary	9	9	9	Pass

15. Algorithm: Smallest Digit

Function: `int smallest_digit(int number)`

1. Start.
2. Initialize `smallest` with the last digit.
3. Remove the last digit.
4. Extract every remaining digit.
5. Compare it with `smallest`.
6. If the digit is smaller, update `smallest`.
7. Repeat for all digits.
8. Return `smallest`.

9. Stop.

Test Cases

Test	Input	Expected	Actual	Result
Valid	58321	1	1	Pass
Valid	9075	0	0	Pass
Boundary	5	5	5	Pass

16. Algorithm: GCD

Function: `int find_gcd(int first, int second)`

1. Start.
 2. Accept two positive integers.
 3. Apply the Euclidean algorithm.
 4. Calculate the remainder of the two numbers.
 5. Replace the numbers appropriately.
 6. Repeat using a `while` loop until the second value becomes zero.
 7. Return the GCD.
 8. Stop.
-

17. Algorithm: LCM

Function: `int find_lcm(int first, int second)`

1. Start.
2. Find GCD using `find_gcd()`.
3. Calculate:

LCM = (first / GCD) × second

4. Return LCM.
5. Stop.

Dividing before multiplication helps reduce the possibility of integer overflow.

GCD & LCM Test Cases

Test	First	Second	Expected GCD	Expected LCM	Actual	Result
Valid	12	18	6	36	GCD=6, LCM=36	Pass
Valid	5	7	1	35	GCD=1, LCM=35	Pass
Boundary	1	1	1	1	GCD=1, LCM=1	Pass

18. Utility Function: Factorial

Function: `int factorial(int number)`

1. Start.
2. Initialize `result = 1`.
3. Start from 2.
4. Multiply `result` by each integer up to `number`.
5. Return `result`.
6. Stop.

Recursion must **not** be used.

19. Utility Function: Integer Power

Function: `int integer_power(int base, int exponent)`

1. Start.
2. Initialize `result = 1`.
3. Multiply `result` by `base`.
4. Repeat `exponent` times.
5. Return `result`.
6. Stop.

The `pow()` function must not be used.

20. HEADER FILE — `number.h`

```
#ifndef NUMBER_H
#define NUMBER_H

#define TRUE 1
#define FALSE 0
```

```

int count_digits(int number);
int sum_of_digits(int number);
int product_of_digits(int number);
int reverse_number(int number);
int largest_digit(int number);
int smallest_digit(int number);

int is_palindrome(int number);
int is_armstrong(int number);
int is_strong(int number);
int is_perfect(int number);
int is_prime(int number);

int factorial(int number);
int integer_power(int base, int exponent);
int find_gcd(int first, int second);
int find_lcm(int first, int second);

#endif

```

The header guard prevents the contents of the header file from being processed more than once. TRUE and FALSE are defined as 1 and 0.

21. digit_operations.c

```

#include "number.h"

int count_digits(int number)
{
    int count = 0;

    for (; number != 0; number /= 10)
        count++;

    return count;
}

int sum_of_digits(int number)
{
    int sum = 0;

    for (; number != 0; number /= 10)
        sum += number % 10;

    return sum;
}

int product_of_digits(int number)
{
    int product = 1;

```

```

        for (; number != 0; number /= 10)
            product *= number % 10;

    return product;
}

int reverse_number(int number)
{
    int reverse = 0, digit;

    for (; number != 0; number /= 10)
    {
        digit = number % 10;
        reverse = reverse * 10 + digit;
    }

    return reverse;
}

int largest_digit(int number)
{
    int largest = number % 10;
    int digit;

    for (number /= 10; number != 0; number /= 10)
    {
        digit = number % 10;

        if (digit > largest)
            largest = digit;
    }

    return largest;
}

int smallest_digit(int number)
{
    int smallest = number % 10;
    int digit;

    for (number /= 10; number != 0; number /= 10)
    {
        digit = number % 10;

        if (digit < smallest)
            smallest = digit;
    }

    return smallest;
}

```

22. number_properties.c

```
#include "number.h"
```

```

int is_palindrome(int number)
{
    if (reverse_number(number) == number)
        return TRUE;

    return FALSE;
}

int is_armstrong(int number)
{
    int original = number;
    int digits = count_digits(number);
    int digit, sum = 0;

    for (; number != 0; number /= 10)
    {
        digit = number % 10;
        sum += integer_power(digit, digits);
    }

    if (sum == original)
        return TRUE;

    return FALSE;
}

int is_strong(int number)
{
    int original = number;
    int digit, sum = 0;

    for (; number != 0; number /= 10)
    {
        digit = number % 10;
        sum += factorial(digit);
    }

    if (sum == original)
        return TRUE;

    return FALSE;
}

int is_perfect(int number)
{
    int divisor, sum = 0;

    for (divisor = 1; divisor <= number / 2; divisor++)
    {
        if (number % divisor == 0)
            sum += divisor;
    }

    if (sum == number)
        return TRUE;
}

```

```

        return FALSE;
    }

int is_prime(int number)
{
    int divisor;

    if (number < 2)
        return FALSE;

    for (divisor = 2;
         divisor <= number / divisor;
         divisor++)
    {
        if (number % divisor == 0)
            return FALSE;
    }

    return TRUE;
}

```

23. number_utils.c

```

#include "number.h"

int factorial(int number)
{
    int i, result = 1;

    for (i = 2; i <= number; i++)
        result *= i;

    return result;
}

int integer_power(int base, int exponent)
{
    int i, result = 1;

    for (i = 1; i <= exponent; i++)
        result *= base;

    return result;
}

int find_gcd(int first, int second)
{
    int remainder;

    while (second != 0)
    {
        remainder = first % second;
        first = second;
        second = remainder;
    }
}

```

```

    }

    return first;
}

int find_lcm(int first, int second)
{
    return (first / find_gcd(first, second)) * second;
}

```

24. MAIN PROGRAM — `main.c`

```

#include <stdio.h>
#include "number.h"

int main()
{
    int number, second;
    int choice;

    do
    {
        printf("Enter a positive integer: ");
        scanf("%d", &number);

        if (number <= 0)
            printf("Invalid input! Enter a positive integer.\n");

    } while (number <= 0);

    do
    {
        printf("\n===== MENU =====\n");
        printf("1. Count the digits\n");
        printf("2. Sum of digits\n");
        printf("3. Product of digits\n");
        printf("4. Reverse the number\n");
        printf("5. Check Palindrome\n");
        printf("6. Check Armstrong number\n");
        printf("7. Check Strong number\n");
        printf("8. Check Perfect number\n");
        printf("9. Check Prime number\n");
        printf("10. Find largest digit\n");
        printf("11. Find smallest digit\n");
        printf("12. Find GCD and LCM\n");
        printf("13. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("Number of digits = %d\n",

```

```

        count_digits(number));
    break;

case 2:
    printf("Sum of digits = %d\n",
        sum_of_digits(number));
    break;

case 3:
    printf("Product of digits = %d\n",
        product_of_digits(number));
    break;

case 4:
    printf("Reversed number = %d\n",
        reverse_number(number));
    break;

case 5:
    if (is_palindrome(number))
        printf("%d is a Palindrome number.\n", number);
    else
        printf("%d is not a Palindrome number.\n", number);
    break;

case 6:
    if (is_armstrong(number))
        printf("%d is an Armstrong number.\n", number);
    else
        printf("%d is not an Armstrong number.\n", number);
    break;

case 7:
    if (is_strong(number))
        printf("%d is a Strong number.\n", number);
    else
        printf("%d is not a Strong number.\n", number);
    break;

case 8:
    if (is_perfect(number))
        printf("%d is a Perfect number.\n", number);
    else
        printf("%d is not a Perfect number.\n", number);
    break;

case 9:
    if (is_prime(number))
        printf("%d is a Prime number.\n", number);
    else
        printf("%d is not a Prime number.\n", number);
    break;

case 10:
    printf("Largest digit = %d\n",
        largest_digit(number));
    break;

```

```

        case 11:
            printf("Smallest digit = %d\n",
                smallest_digit(number));
            break;

        case 12:
            do
            {
                printf("Enter another positive integer: ");
                scanf("%d", &second);

                if (second <= 0)
                    printf("Invalid input!\n");

            } while (second <= 0);

            printf("GCD = %d\n",
                find_gcd(number, second));

            printf("LCM = %d\n",
                find_lcm(number, second));
            break;

        case 13:
            printf("Exiting program...\n");
            break;

        default:
            printf("Invalid menu choice!\n");
    }

    } while (choice != 13);

    return 0;
}

```

This follows the specified structure: validate the positive integer, use a `do-while` menu, obtain the user's choice, and use `switch-case` to call the appropriate function.

25. Compilation

Since this is a **modular C project**, all `.c` files need to be compiled together.

```
gcc main.c digit_operations.c number_properties.c number_utils.c -o
experiment7
```

Then execute:

```
./experiment7
```

26. Sample Testing

```
(kali㉿kali)-[~/Desktop]
$ gcc main.c digit_operations.c number_properties.c number_utils.c -o experiment7

(kali㉿kali)-[~/Desktop]
$ ./experiment7
Enter a positive integer: 153

===== MENU =====
1. Count the digits
2. Sum of digits
3. Product of digits
4. Reverse the number
5. Check Palindrome
6. Check Armstrong number
7. Check Strong number
8. Check Perfect number
9. Check Prime number
10. Find largest digit
11. Find smallest digit
12. Find GCD and LCM
13. Exit
Enter your choice: 6
153 is an Armstrong number.
```

```
13: Exit
Enter your choice: 1
Number of digits = 3

Enter your choice: 2
Sum of digits = 9

Enter your choice: 3
Product of digits = 15

Enter your choice: 4
Reversed number = 351

Enter your choice: 5
153 is not a Palindrome number.

Enter your choice: 7
153 is not a Strong number.

Enter your choice: 8
153 is not a Perfect number.

Enter your choice: 9
153 is not a Prime number.

Enter your choice: 10
Largest digit = 5

Enter your choice: 11
Smallest digit = 1

Enter your choice: 12
Enter another positive integer: 351
GCD = 9
LCM = 5967

Enter your choice: 13
Exiting program...
```

27. Result

The menu-driven modular C program was designed using **functions, loops, decision-making statements, header files, and multiple source files**. The program performs all the required mathematical and digit-based operations, and the prepared test cases verify the expected behaviour of the functions.

28. Conclusion

The experiment demonstrates the use of **modular programming in C** by dividing a large program into smaller functions and separate source files. Iterative constructs such as `for`, `while`,

and `do-while` were used along with `if-else` and `switch-case` decision-making constructs. The program was tested using valid, invalid, and boundary inputs as required by the experiment.